

=== SAKUM LANG: A PURE ASSEMBLY AGENTIC AI SYSTEM ===

PART 1: CORE PHILOSOPHY

Sakum is an agentic AI runtime written entirely in x86-64 assembly
No libc, no runtime, no dependencies -- raw BSD syscalls only
Inspired by human immune system: danger theory + self-healing
Survivability metric $S(t) = \text{integral of health over time}$

PART 2: THREE-LAYER PROTECTION MODEL

Layer 1 -- Hardware: CPU rings, memory protection, NX bit
Layer 2 -- OS: macOS sandbox, pledge/unveil, syscall filtering
Layer 3 -- Agent: Decision gates, capability tokens, audit log
Agent can NEVER override Layer 1 or Layer 2 decisions

PART 3: KEY COMPONENTS

serve.s -- HTTP server, request routing, capability enforcement
health.s -- Continuous self-monitoring, anomaly detection
relay.s -- Inter-agent messaging, capability delegation
monitor.s -- Metrics collection, survivability scoring
pdf.s -- Pure assembly PDF generation (this document)

PART 4: SELF-HEAL CYCLE

1. DETECT: Health checks, heartbeat timeouts, invariant violations
2. DIAGNOSE: Root cause isolation via dependency graph
3. HEAL: Restart component, rollback state, re-route traffic
4. VERIFY: Post-heal health check, metric regression test

PART 5: DEVELOPMENT WORKFLOW

Write .s files in assembly/sakum/ directory
Build: `gcc -arch x86_64 assembly/*.s -o sakum`
Run: `./sakum [args]` -- single binary, zero dependencies
Test: Assembly-level unit tests in test/*.s

PART 6: SAFETY GUARANTEES

Memory safety: No heap allocator -- stack + static only
No code injection: W^X enforced, no JIT, no eval
Capability-based: Every action requires explicit token
Audit trail: Every syscall logged with timestamp + decision

PART 7: EXTENDING SAKUM

Add new agents by implementing the Agent interface in .s
Register capabilities in capability.def at build time
Health checks auto-discovered via section naming convention
Survivability dashboard at /metrics endpoint

PART 8: GETTING STARTED

1. Clone repo, cd to root
2. make build (produces single sakum binary)
3. ./sakum serve -- starts HTTP on :8080
4. curl localhost:8080/health -- verify survivability > 0.95
5. Read assembly/serve.s to understand request flow

SEE ALSO: docs/ARCHITECTURE.md, docs/SAFETY.md, docs/API.md